

Auditoría integral de implementación

Proyecto: Sistema de Orquestación de Atención al Cliente Bancario

Documento contrastado: TG1_ChristianMendoza.pdf, 100 páginas

Fecha de corte: 17 de julio de 2026, zona America/La_Paz

Alcance: backend, agentes, privacidad, RAG, persistencia, frontend, Docker, configuración, pruebas, dependencias y operación demostrativa.

Dictamen ejecutivo

El sistema quedó **alineado funcionalmente con el documento dentro del alcance de prototipo académico**. De los 17 requerimientos funcionales, 16 se verifican integralmente y RF-05 queda como cumplimiento parcial controlado por una limitación inherente al canal de voz: el audio se envía al servicio de transcripción antes de que el backend pueda enmascarar la transcripción.

De los 15 requerimientos no funcionales, 12 se verifican dentro del entorno controlado y 3 quedan parciales: privacidad extremo a extremo del audio, medición formal de rendimiento y certificación formal de accesibilidad.

El resultado **no debe presentarse como software bancario productivo**. No autentica clientes reales, no consulta información financiera, no ejecuta operaciones y no integra un core bancario.

Hallazgos encontrados en la línea base

Antes de la corrección se encontraron estas brechas:

1. El frontend mostraba datos estáticos y temporizadores simulados; no consumía el backend ni completaba RF-01, RF-11 o RF-12 de extremo a extremo.
2. Existían datos mock duplicados en el frontend y dos configuraciones Compose separadas.
3. Las migraciones dependían de metadata viva de SQLAlchemy, lo que hacía que una migración histórica pudiera cambiar cuando cambiaban los modelos.
4. La evaluación RAG daba por aprobados casos donde la respuesta automática estaba prohibida sin ejecutar realmente la política de clasificación.
5. Los PDF de conocimiento podían generarse desde código y coexistían con el manifiesto, creando dos fuentes de verdad.
6. La derivación inicial filtraba de manera demasiado amplia y su componente semántico partía de una puntuación constante; la similitud no podía alterar el ranking.
7. Faltaban expiración efectiva de sesión de kiosco, validación estricta de estados y persistencia de la prioridad propuesta.
8. No existía administración gerencial del ciclo documental.
9. Había credenciales de desarrollo como respaldo dentro del objeto de configuración.
10. El frontend no tenía imagen Docker ni configuración integral del sistema.

Correcciones implementadas

- Frontend real para kiosco, ejecutivo y gerencia; se eliminó mock-data.ts.
- Conversación speech-to-speech por WebRTC con secreto efímero, VAD semántico, interrupciones, subtítulos en memoria y tools delegadas al backend.

- Proxy de mismo origen en Next.js para API, cookies, multipart y descargas.
- Access token solo en memoria, refresh Ht tpOn l y rotado y guardado como hash.
- Orquestador con máquina de estados, expiración, idempotencia y control de

aclaración.

- Clasificación general/personalizada/sensible y política conservadora de fallback.
- Priorización por categoría, criticidad, urgencia, seguridad, angustia y preferencia.
- Derivación por categoría exacta, similitud semántica, experiencia y carga activa.
- RAG limitado a consultas generales, con umbral, vigencia, citas verificadas y

fallback humano.

- Migraciones Alembic explícitas y congeladas, incluida pgvector/HNSW.
- Gestión documental: alta, edición, versión, descarga, reindexación, activación y

archivo.

- Validación de PDF, ruta UUID, MIME, tamaño, páginas, texto, categorías y URL de fuente HTTP(S).

- Semilla declarativa en backend/seed/demo_seed . j son.
- Un solo Compose raíz y una sola imagen backend compartida por API, migración e ingesta.

- Identidad de despliegue, orígenes y configuración sensible obligatorios desde . env, sin datos operativos ni credenciales de respaldo.

- Encabezados de seguridad del frontend y contenedores sin privilegios.

Matriz de requerimientos funcionales

ID	Estado	Evidencia y criterio de auditoría
RF-01	Cumple	kiosk-provider . tsx mantiene una sesión Realtime speech-to-speech continua con VAD, interrupciones y subtítulos; openai_provider . py crea el secreto efímero.
RF-02	Cumple	La tool aplica una guarda determinista de sí/no explícito y delega en orchestrator . py, que conserva la decisión de negocio y confirma o retorna CAPTURE para corrección.
RF-03	Cumple	ClassificationAgent produce una de las cinco categorías definidas en enums . py.
RF-04	Cumple	El orquestador emite CLARIFY, conserva contexto y limita intentos mediante MAX_CLARIFICATIONS.
RF-05	Parcial controlado	PIIMaskingService elimina correo, tarjeta, cuenta, teléfono, identificador, monto y nombre antes de clasificación/persistencia. El audio ya pasó por el transcriptor externo; ver riesgos residuales.
RF-06	Cumple	PrioritizationAgent evalúa tipo, urgencia, incidente, angustia y atención preferente; la prioridad se persiste.
RF-07	Cumple	Fraude/movimiento no reconocido resulta crítico; bloqueo resulta alto y las señales de seguridad refuerzan el tratamiento.
RF-08	Cumple	InitialAttentionAgent intenta RAG solo en nivel general; sin evidencia crea ticket humano.
RF-09	Cumple	DerivationAgent exige habilidad compatible y pondera semántica, experiencia y carga. Una prueba específica demuestra que la semántica puede prevalecer sobre experiencia.
RF-10	Cumple	models . py persiste sesión, requisito, caso, ticket y eventos; tickets . py expone detalle y ciclo de estados.
RF-11	Cumple	frontend/app/ejecutivo/page . tsx y el detalle muestran asignación, resumen, prioridad, estado y trazas reales.
RF-12	Cumple	frontend/app/gerencial/page . tsx consume métricas y casos filtrados desde management . py.
RF-13	Cumple	El resultado incluye ticket e información de seguimiento/reclamo; la base RAG contiene canales documentados.
RF-14	Cumple	La sesión registra preferential_attention; el priorizador eleva solo prioridades bajas/medias sin degradar urgentes.

ID	Estado	Evidencia y criterio de auditoría
RF-15	Cumple	Los niveles personalizado y sensible retornan IDENTIFY; el frontend advierte que solo se usan datos ficticios.
RF-16	Cumple	El identificador normalizado se compara mediante HMAC con clientes simulados; se registra éxito o fallo sin guardar el valor completo.
RF-17	Cumple	Solo nivel general admite automatización. Personalizado y sensible requieren identificación y terminan en ticket humano.

Matriz de requerimientos no funcionales

ID	Estado	Evidencia y límite
RNF-01	Cumple	Flujo conversacional guiado, confirmación por voz, subtítulos, estados visibles y controles de recuperación ante errores.
RNF-02	Parcial controlado	Minimización, hash, masking y autorización están implementados; el audio sin enmascarar llega al proveedor Realtime.
RNF-03	Cumple	Roles EXECUTIVE/MANAGER, JWT, sesión de kiosco opaca y control de pertenencia de tickets. Un ejecutivo obtiene 403 en conocimiento gerencial.
RNF-04	Cumple	Eventos por captura, masking, clasificación, prioridad, identificación, ruta y estado; RAG registra IDs, resultado y hash de respuesta.
RNF-05	Cumple en prototipo	Healthchecks, dependencias condicionadas, reintento de voz/fallback humano y procedimiento explícito de arranque/reinicio. Por decisión operativa, los contenedores no arrancan con la computadora. No equivale a alta disponibilidad.
RNF-06	Parcial	Async I/O, timeout, batch de embeddings, HNSW, top-k y paginación. No existe todavía una prueba de carga con SLO formal.
RNF-07	Cumple	API, dominio, servicios, repositorios, agentes y conocimiento tienen límites explícitos.
RNF-08	Cumple en prototipo	Categorías/reglas/modelos son configurables y las migraciones permiten evolución. Añadir una categoría sigue requiriendo cambio de enum, UI y migración.
RNF-09	Cumple	Tipado, lint, pruebas, configuración central, manifiesto y documentación operativa.
RNF-10	Parcial	Voz, subtítulos, etiquetas, estados, identificación escrita y varios atributos ARIA están presentes. Falta auditoría WCAG con lector de pantalla/contraste automatizado.
RNF-11	Cumple	Datos ficticios, advertencias explícitas y ausencia total de integración core.
RNF-12	Cumple	Métricas, trazas, matriz RF/RNF, evaluación RAG y reporte reproducible.
RNF-13	Cumple	Identificador HMAC, valor visible parcialmente enmascarado y ninguna persistencia de audio/transcripción original.
RNF-14	Cumple	UI, voz y dominio distinguen identificación demostrativa de autenticación bancaria.
RNF-15	Cumple	Ejecutivo ve solo sus tickets y sesión enmascarada; gerencia ve agregados sin identificadores ni PII.

Evidencia de backend y agentes

La implementación principal se encuentra en:

- `backend/app/services/orchestrator.py`:

máquina de estados y coordinación.

- `backend/app/services/agents.py`: cuatro agentes.
- `backend/app/services/pii.py`: masking local.
- `backend/app/knowledge/service.py`:

recuperación y grounding.

- `backend/app/knowledge/management.py`:

ciclo de PDF.

- `backend/app/api/knowledge.py`: API gerencial.
- `backend/app/db/models.py`: persistencia.
- `backend/alembic/versions`: historia de esquema.

El backend no es una arquitectura distribuida de agentes autónomos; es un **monolito modular con agentes especializados coordinados por un orquestador**, coherente con el alcance y más demostrable para un prototipo.

Evidencia de frontend

- `frontend/components/providers/auth-provider.tsx`:

sesión y renovación.

- `frontend/components/providers/kiosk-provider.tsx`:

estado, sesión Realtime, tools, reconexión, privacidad y cierre del kiosco.

- `frontend/lib/kiosk-realtime.ts`:

instrucciones, esquemas de tools y subtítulos en memoria.

- `[frontend/app/backend-api/[...path]/route.ts](../frontend/app/backend-api/[...path]/route.ts)`:

proxy de mismo origen.

- `frontend/app/kiosco`: flujo completo.
- `frontend/app/ejecutivo`: operación.
- `frontend/app/gerencial`: métricas y conocimiento.

No quedan colecciones mock en el código de aplicación. Las etiquetas visuales centralizadas en `frontend/lib/labels.ts` son constantes de presentación, no datos operativos.

Evidencia Docker y configuración

- `docker-compose.yml` configura cinco servicios y dos

volúmenes.

- `backend/Dockerfile` usa Python 3.12 y usuario sin

privilegios.

- `frontend/Dockerfile` usa build multietapa, salida

standalone y usuario sin privilegios.

- `.env.example`,

`backend/.env.example` y `frontend/.env.example` documentan la configuración.

La API key de OpenAI existe únicamente en el entorno del backend. El navegador recibe un secreto efímero de Realtime asociado a su sesión, nunca la API key del servidor.

Pruebas ejecutadas

Resultado del corte:

- `uv run ruff check .`: aprobado.
- `uv run pytest -q`: **26 pruebas aprobadas**.
- `pnpm test`: **15 pruebas aprobadas**.
- `pnpm lint` y `pnpm typecheck`: aprobados.
- `pnpm build`: aprobado, 13 rutas.
- `pnpm audit --prod`: sin vulnerabilidades conocidas.
- `uv run --with pip-audit pip-audit`: sin vulnerabilidades conocidas.
- `docker compose config --quiet`: aprobado.
- Construcción de imágenes `backend/frontend`: aprobada.

- Arranque Docker: PostgreSQL y backend saludables; migración/bootstrap con código 0.
- Bootstrap real: 8 documentos, 8 fragmentos y 8 perfiles de habilidad.

Pruebas integrales reales ejecutadas por el proxy Next.js:

1. Login gerencial, /auth/me, rotación de refresh y logout.
2. Login ejecutivo y denegación 403 al módulo de conocimiento.
3. Secreto Realtime válido para el modelo configurado.
4. Consulta general: CONSULTA_GENERAL, GENERAL, RAG GROUNDED, dos citas,

ticket automático cerrado.

5. Reporte con número de tarjeta: PII TARJETA, nivel SENSIBLE, prioridad

CRITICO, identificación ficticia exitosa y derivación a un especialista.

6. Ejecutivo: inicio/cierre de atención y conflicto optimista 409 con versión vieja.
7. Gerencia: métricas y listado reflejaron ambos casos.
8. Conocimiento: carga, edición, descarga, nueva versión, desactivación anterior,

reindexación y archivado.

Las pruebas unitarias usan proveedores falsos para ser deterministas; las pruebas integrales anteriores usaron la configuración real autorizada.

Redundancia y mantenibilidad

Se eliminaron:

- frontend/lib/mock-data.ts;
- los controles manuales de grabar/detener, la síntesis de voz del navegador y la

ruta visual redundante de confirmación;

- el generador de corpus y el generador PDF en tiempo de ejecución;
- el Compose duplicado del backend;
- componentes/recursos de plantilla sin uso;
- referencias al comando obsoleto build-pdfs;
- credenciales predeterminadas en código;
- imágenes Docker backend duplicadas por servicio.

Se centralizaron API, tipos, etiquetas, autenticación, configuración pública y estado de kiosco. La semilla y el manifiesto documental son las únicas fuentes declarativas de sus respectivos datos.

Riesgos residuales y recomendaciones

1. **Audio y proveedor externo.** OpenAI recibe audio/transcripción antes del masking

local. Para producción se necesitan consentimiento explícito, evaluación de privacidad, contrato de tratamiento, retención configurada y aprobación legal.

2. **PII heurística.** Las expresiones regulares cubren los datos del prototipo, pero

no garantizan detectar toda PII ni todos los formatos bolivianos. Añadir evaluación con corpus anonimizado y, si corresponde, NER local.

3. **Contenido documental.** Los PDF de demostración no constituyen política oficial

del banco. Gerencia debe cargar fuentes aprobadas, fechas de revisión y responsables de gobierno documental.

4. **Disponibilidad.** Compose brinda continuidad local, no alta disponibilidad. Faltan TLS de borde, backup probado, monitoreo, alertas y recuperación ante desastre.
5. **Rate limiting.** El límite actual está en memoria por proceso. En múltiples réplicas debe migrar a gateway/Redis.
6. **Rendimiento.** Definir SLO y ejecutar carga concurrente para clasificación, RAG, dashboards y cargas de PDF.
7. **Accesibilidad.** Ejecutar WCAG 2.2 AA, navegación solo teclado, lector de pantalla, contraste y pruebas con usuarios.
8. **Autenticación interna.** Los usuarios del personal son de demostración. Integrar un IdP corporativo con MFA antes de un entorno real.
9. **Supabase.** SUPABASE_URL y service-role key no son cadenas PostgreSQL. Deben configurarse DATABASE_URL/DATABASE_MIGRATION_URL, SSL y pooler.
10. **Alcance académico.** Mantener bloqueada cualquier consulta de saldo, movimiento, operación o dato real hasta contar con arquitectura, controles y autorización bancaria formal.

Conclusión

El código permite demostrar las preguntas centrales del documento: cómo se captura, protege, clasifica, prioriza, aclara, identifica, responde, deriva, registra y visualiza un caso. La trazabilidad está presente tanto en código como en pruebas y datos.

El sistema está listo para **demostración funcional controlada**. Los tres puntos parciales no son fallas ocultas: están documentados y delimitan con precisión el trabajo necesario para evolucionar desde prototipo académico hacia un producto con exigencias bancarias reales.